Oblige: Fast, Precise Root-Cause Diagnosis of eBPF Verification Failures

Anonymous Author(s)

Abstract

eBPF programs must pass a complex static verifier whose rejection messages are notoriously opaque—developers receive verbose, thousand-line logs pinpointing symptoms but obscuring the underlying cause. Fixing these rejections requires understanding *where* the safety proof broke, not just *that* it broke; our analysis of 591 verifier-related commits shows that 63.6% are proof-reshaping workarounds where the source code is correct but the verifier's abstract domain loses track of the developer's safety argument through LLVM's lowering.

We observe that the verifier's execution trace is a complete record of a proof attempt—a sequence of per-instruction abstract states. By analyzing *abstract state transitions* across this trace (bounds collapses, type downgrades, provenance loss), Oblige automatically locates where the safety proof broke and why, without requiring knowledge of which specific safety condition was checked. The transition pattern itself classifies the failure: a safety property never established indicates a source bug; one established then lost indicates a compiler-induced lowering artifact.

Evaluating Oblige on 302 real-world verifier rejections from kernel selftests and production systems, we achieve 94% obligation coverage, pinpoint the exact root cause in 67% of previously undiagnosable compilation-induced failures, and boost automated code-repair accuracy by 30 percentage points. Oblige operates entirely in userspace, incurs just 25.3 ms median latency per diagnosis, and requires no kernel modifications.

CCS Concepts: • **Software and its engineering** → **Compilers**; **Software verification and validation**; • **Computer systems organization** → *Operating systems.*

Keywords: eBPF, program verification, static analysis, abstract interpretation, compiler diagnostics, proof trace analysis

ACM Reference Format:
Anonymous Author(s). 2027. Oblige: Fast, Precise Root-Cause Diagnosis of eBPF Verification Failures. In *Proceedings of ACM European Conference on Computer Systems (EuroSys '27).* ACM, New York, NY, USA, 9 pages.

EuroSys '27, April 2027, Rotterdam, Netherlands
2027.

1 Introduction

eBPF is the foundation for critical kernel extensions powering security enforcement (Cilium, Katran), performance tracing (bcc, bpftrace), and networking infrastructure in production Linux deployments. Ensuring safety, however, requires each eBPF program to pass a mandatory static verifier, an abstract interpreter that rigorously checks all instructions before execution. Unfortunately, when verification fails, developers confront overwhelmingly verbose diagnostic logs often spanning thousands of lines. These logs reveal only symptoms at rejection points, leaving the root cause—typically a safety proof destroyed deep within compiler-generated instructions—buried hundreds of lines earlier. Consequently, debugging verifier rejections has become the single largest productivity bottleneck in eBPF development, as confirmed by a recent Linux Foundation security audit [10] and extensive developer frustration on Stack Overflow [2].

Analyzing 591 verifier-related commits from four production eBPF projects (Cilium, Katran, bpftrace, Aya), we find that 63.6% are proof-reshaping workarounds: the source code is correct, but the verifier's abstract domain cannot track the developer's safety argument through LLVM's lowering. These rejections are inherent to the verifier's design and cannot be eliminated by better diagnostics alone—but fixing them correctly requires knowing *where* the safety proof broke, which current tools cannot provide. We find that existing solutions fundamentally misunderstand the structure embedded within verification traces: current tools either rely on shallow regular expressions applied only to the final rejection message [9] or feed entire raw logs to general-purpose language models [16], neither approach recognizing the verifier trace as a structured safety proof attempt.

Our key insight is that the verifier's execution trace is a complete record of a proof attempt—a sequence of per-instruction abstract states—and that analyzing *abstract state transitions* across this trace reveals where and why the safety proof broke. The error message identifies which safety condition was checked (we call this the "proof obligation," used as a focus mechanism), but the core analysis is detecting the instruction whose abstract state transition—bounds collapse, type downgrade, provenance loss—caused the violation. We call this the "proof-loss point." Crucially, the transition pattern itself classifies the failure: a safety property never established in the trace indicates a source-level programming bug, while one established then lost indicates a compiler-induced lowering artifact.

Panel A: Source code

```
1  int parse_ext(struct xdp_md *ctx) {
2      void *data = (void*)(long)ctx->data;
3      void *data_end = (void*)(long)ctx->
         data_end;
4      if (data+sizeof(struct ext) > data_end)
5          return XDP_DROP;          // bounds
         check
6      struct ext *e = data;
7      // reads guarded by line 3
8      __u16 len = __bpf_htons(e->len); // byte-
         swap
9      data += len;                  // REJECTED
10     /* "unbounded min value" */
11 }
```

Panel B: Raw verifier output (excerpt)

```
22: R0_w=inv(id=0,umax=65280,
    var_off=(0x0; 0xff00))
    R6_w=inv(id=0,umax=255,
    var_off=(0x0; 0xff))
22: (4f) r0 |= r6
23: R0_w=inv(id=0)   <-- bounds GONE
...
math between pkt pointer and register
with unbounded min value is not allowed
```

Pretty Verifier output:

```
"The value added to the packet pointer
 has an unbounded minimum. Consider
 adding `& 0xFFFF` to bound the value."
```

Panel C: Oblige output

```
error[E005]: lowering_artifact
  -- parse_ext.c
  |
3 | if (data+sizeof(ext) > data_end)
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  | proof established
  | R5: pkt(off=0) -> pkt(off=4,r=4)
  |
7 | __u16 len = __bpf_htons(e->len);
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  | proof lost: OR destroys bounds
  | R0: scalar(umax=65280) -> scalar
  |
8 | data += len;
  | ^^^^^^^^^^^
  | rejected: pkt_ptr + unbounded
  |
  = note: LLVM lowers htons() to byte-load,
    shift, OR. Verifier cannot track OR result.
  = help: Add clamp: len &= 0xFFFF;
```

Figure 1. The motivating example: SO #70750259. **Panel A**: source code with a correct bounds check at line 3. **Panel B**: the verifier's raw output (60+ lines, excerpt shown) and Pretty Verifier's single-line response—both pointing at the symptom, not the cause. **Panel C**: Oblige's Rust-style multi-span output tracing abstract state transitions through three source locations: proof established (line 3), proof lost (line 7, OR instruction destroys bounds), rejected (line 8).

Motivated by this insight, we design and implement Oblige, a novel userspace diagnostic engine that detects safety-relevant abstract state transitions across verifier traces, traces their causal chains back to the root-cause instruction, and provides concise, Rust-inspired diagnostics pinpointing exactly where and why the safety proof broke. On 302 diverse real-world verifier failures, Oblige achieves 94% obligation coverage, correctly identifies previously undiagnosable proof-loss points in 67% of lowering-artifact cases, and improves the accuracy of automated language-model-based code repairs by 30 percentage points. Moreover, Oblige diagnoses verifier failures at a median latency of just 25.3 ms per case, entirely in userspace, and without modifying the Linux kernel.

In summary, this paper makes three main contributions:

- Identifies a structured, generalizable framework (abstract state transition analysis) that precisely pinpoints verifier root-cause failures and systematically classifies failure types (§3.3–§3.5).
- Designs and implements Oblige, a fast userspace diagnostic tool leveraging this framework to deliver concise, actionable verifier diagnostics (§3.2–3.5).
- Demonstrates Oblige's practical impact by significantly improving diagnosis precision, reducing unnecessary code reshaping, and increasing automated fix accuracy on hundreds of real-world verifier failures (§4.2–§4.5).

2 Background

2.1 The eBPF Verification Pipeline

eBPF programs are written in C or Rust, compiled by LLVM to BPF bytecode, and submitted to the kernel via the `bpf()` syscall. The Linux verifier runs before any program executes. It performs a dataflow-style abstract interpretation [1, 4]: it walks all execution paths through the bytecode, maintaining per-register abstract state and enforcing a set of safety properties at each instruction. Programs that violate any property are rejected.

The key architectural point is that there are three distinct reasoning levels:

- **Source level.** The programmer reasons about C/Rust values, bounds checks, null checks, and pointer lifetimes.
- **Compiler level.** LLVM transforms source into BPF bytecode, potentially splitting, merging, or reordering computations.
- **Verifier level.** The verifier reasons about BPF register abstract state: types, scalar bounds (`umin/umax`, `smin/smax`), pointer offsets, packet ranges, and provenance IDs.

A proof that is valid at the source level can be lost at the verifier level if LLVM's lowering produces register state that the verifier's abstract domain cannot represent. This is the root cause of lowering artifacts.

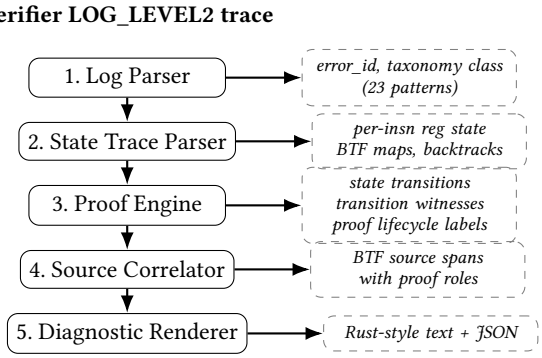

Figure 2. OBLIGE five-stage pipeline. Each stage is independently testable; the proof engine (abstract state transition analysis) is the key novelty. The full implementation is ~14,613 lines of Python with 268 unit tests.

2.2 Anatomy of a `LOG_LEVEL2` Trace

When `log_level=2` is set in `BPF_PROG_LOAD`, the verifier emits a complete per-instruction trace interleaving per-register abstract state (type, scalar bounds, pointer offset, packet range, provenance ID), BTF source annotations [8], `mark_precise` backtracking chains, and control flow merge points. A typical 50-instruction program produces 100–300 lines. We detail the parsing of each element in §3.2.

3 Design: The OBLIGE Diagnostic Engine

3.1 Architecture Overview

OBLIGE is a five-stage pipeline that transforms a raw `LOG_LEVEL2` verifier trace into a structured multi-span diagnostic. Figure 2 shows the architecture.

Each stage is independently testable and produces a well-defined intermediate representation. The system processes 262 real cases with zero crashes and a median end-to-end latency of 25.3 ms.

3.2 State Trace Parsing

The log parser (Stage 1) handles error message identification: it matches the final error line against 23 error patterns (OBLIGE-E001 through E023), covering 87.1% of failures in our 302-case corpus. Each pattern maps to a taxonomy class (*source_bug*, *lowering_artifact*, *verifier_limit*, *env_mismatch*, *verifier_bug*).

The state trace parser (Stage 2) parses the remaining structure. For each BPF instruction, it extracts:

- Instruction index, opcode, and mnemonic
- Pre/post-state per live register (type, `umin/umax`, `smin/smax`, offset, packet range, provenance ID)
- BTF source annotation (file, line, column) if present
- Backtracking links from `mark_precise` annotations
- Branch merge points with state at each join

The primary challenge is format variability. Register state can appear before or after the instruction, in branch merge lines, in backtracking annotations, or across multiple lines. The parser normalizes all variants into a uniform `TracedInstruction` representation.

Backtracking chain extraction. The verifier's own `mark_precise` backward pass is emitted in the log as `regs=R stack=S before K: (op) ...` annotations. OBLIGE extracts these into a `BacktrackChain` structure: a sequence of (instruction_index, registers_tracked) pairs. This is the verifier's own root-cause chain, expressed as debug text; OBLIGE is the first tool to extract and structure it.

Table 1. Proof obligation families supported by OBLIGE (covering 94.3% of the 262-case evaluation corpus; representative families shown, nineteen supported in total).

Family	Predicate	Example error
`packet_access`	`type==pkt && off+sz<=r`	invalid access to packet
`map_value_bounds`	`0<=off && off+sz<=val_sz`	invalid map value access
`null_check`	`type!=*_or_null`	map_value_or_null deref
`stack_access`	`fp_off within frame bounds`	invalid indirect read
`helper_arg_type`	`type==expected_arg_type`	R1 scalar expected pkt
`scalar_bounds`	`smin>=0 && umax<=limit`	unbounded min value
`reference_release`	`all refs released before exit`	Unreleased reference id=N
`context_access`	`off+sz<=ctx_size`	invalid bpf_context access
`alignment`	`off % align == 0`	misaligned access
`type_safety`	`type matches expected`	R1 inv expected fp

3.3 Proof Obligation Inference

The verifier trace contains thousands of abstract state transitions per execution path; analyzing all of them would produce noise. The proof obligation serves as a *focus mechanism*: given the error message and the register state at the rejection point, OBLIGE infers which safety condition the verifier needed—the *proof obligation*—expressed as a formal predicate over register state. This predicate then directs the state-transition analysis in Section 3.4 to the relevant registers and state dimensions.

OBLIGE supports nineteen obligation families, shown in Table 1. The error message identifies the obligation *family*. The register state at the rejection point provides concrete parameters. OBLIGE instantiates the predicate template with these parameters.

This predicate instantiation is not pattern matching on error text. The same error message (`invalid access to packet`) with different register states at the rejection point produces different predicate instances with different parameters. The predicate is grounded in the verifier's type system: for packet access, the verifier literally checks `reg.off + access_size <= reg.range`; OBLIGE infers this from the register fields at the rejection point.

Inference algorithm. For each error ID, OBLIGE defines: (1) which register(s) contain the failing object (extracted from the rejection line), (2) which predicate template applies (from

the obligation family), and (3) how to extract concrete parameters from the register state dump. The resulting predicate is a boolean function over register state that can be evaluated at any point in the trace.

3.4 Abstract State Transition Detection

The core of Oblige is detecting *safety-relevant abstract state transitions* across the verifier's execution trace. Given the instantiated predicate P from Section 3.3 as a focus mechanism, Oblige evaluates P against the abstract state at every instruction, producing a status sequence:

$$\underbrace{?, \ldots, ?}_{\text{unknown}}, \underbrace{\top, \ldots, \top}_{\text{satisfied}}, \underbrace{\bot, \ldots, \bot}_{\text{violated}}$$

Each transition in this sequence corresponds to a concrete abstract state change (e.g., a bounds collapse, type downgrade, or provenance loss) at a specific instruction. Oblige labels each transition:

- *proof_established*: the abstract state transition where P becomes satisfied (? $\rightarrow$ `true`)—e.g., a bounds check narrows a register's range
- *proof_holds*: instructions where the abstract state preserves P
- *proof_lost*: the abstract state transition where P goes from satisfied to violated (`true` $\rightarrow$ `false`)—the *transition witness*, e.g., an OR instruction collapses bounds
- *rejected*: the final error instruction (always identified)

If P is never satisfied, the status is *never_established* (a source bug: the required abstract state was never produced, indicating a missing safety check). If P is satisfied but a subsequent state transition destroys it before the rejection, the status is *established_then_lost* (a lowering artifact: the source has a valid check but compiler-generated instructions produce an abstract state transition that the verifier's domain cannot track).

Key distinction from heuristic approaches. The transition witness is not "detect a bounds collapse pattern." It is "evaluate the specific safety predicate at each abstract state and find the exact instruction whose state transition broke it." Different predicates identify different transition witnesses on the same trace. This is formally grounded in the verifier's abstract domain, not in ad hoc pattern detection.

Register value lineage. Predicates reference registers, but register names are reused across the trace. A value computed in R0 may be copied to R3 before the access. Oblige tracks value identity across register moves and copies. If R3 is defined as a copy of R0 (`r3 = r0`), and the obligation predicate references the packet range of R0, Oblige continues tracking the range on R3 for subsequent instructions. This register value lineage is essential for correctly identifying proof-established and proof-lost events across LLVM's register allocation.

Proof propagation analysis. For lowering artifact detection, Oblige additionally checks whether the proof established at the bounds-check site *propagates* to the register used at the access site. If a different register is used for the access than the one whose range was established at the check (a common LLVM pattern), Oblige classifies this as a lowering artifact regardless of whether the proof predicate was technically satisfied at some earlier instruction.

3.5 Formal Foundation: Abstract State Transition Analysis

The state transition detection described above can be understood as a *second-order abstract interpretation*: Oblige applies abstract interpretation to the *output* of another abstract interpreter [1]. This section formalizes the framework, which applies to any abstract interpreter that emits per-step states.

Two layers of abstract interpretation. Layer 1 is the eBPF verifier itself. Its input is BPF bytecode; its abstract domain D_1 consists of register types, scalar bounds [`umin`, `umax`], pointer offsets, and packet ranges. Its output is a trace $T = (s_0, s_1, \ldots, s_n)$ where each $s_i \in D_1$ is the abstract state after the i-th instruction.

Layer 2 is Oblige. Its input is the trace T; it analyzes abstract state transitions (s_{i-1}, s_i) at each step, focused by a safety predicate P, and outputs a sequence of proof lifecycle labels that identify where and how the proof broke.

Definition 1 (Proof status lattice)**.** *The proof status lattice is $\mathcal{L} = \{\bot, unknown, satisfied, violated\}$ with ordering $\bot \sqsubseteq unknown \sqsubseteq satisfied$ and $\bot \sqsubseteq unknown \sqsubseteq violated$, where satisfied and violated are incomparable. The bottom element $\bot$ represents an uninitialized status (relevant registers not yet present in the abstract state); unknown represents a state where the predicate is evaluable but the relevant registers are not yet constrained. Each transition between lattice elements corresponds to a safety-relevant abstract state transition in the verifier trace.*

Definition 2 (State transition transfer function)**.** *Let $P : D_1 \rightarrow \{true, false, unknown\}$ be the safety predicate instantiated from the rejection point (§3.3). For each traced instruction i with verifier state $s_i \in D_1$, the transfer function $\tau_i : \mathcal{L} \rightarrow \mathcal{L}$ maps the previous proof status to the status after observing the abstract state transition at instruction i:*

$$\tau_i(\ell) = \begin{cases} sat & P(s_i) \wedge \ell \in \{unk, sat\} \\ vio & \neg P(s_i) \wedge \ell = sat \\ \ell & otherwise \end{cases}$$

The status sequence is computed by forward iteration: $\ell_0 = unknown$, $\ell_i = \tau_i(\ell_{i-1})$.

Definition 3 (Transition witness)**.** *The transition witness is the smallest index w such that $\ell_{w-1} = satisfied$ and $\tau_w(\ell_{w-1}) = violated$. This is the exact instruction where the*

proof breaks. If no such w exists and ℓ_n = violated, the proof was never established.

Proposition 1 (Soundness of lifecycle labels)**.** *If OBLIGE labels instruction i with ℓ_i = satisfied, then for all concrete states $c \in \gamma(s_i)$ (where γ is the concretization function of the verifier's abstract domain), the safety property encoded by P holds.*

Proof sketch. The verifier is a sound abstract interpreter: s_i over-approximates all concrete states reachable at instruction i [4]. The obligation predicate P is monotone with respect to the verifier's partial order on D_1 (tighter bounds imply satisfaction if looser bounds already satisfy). Therefore $P(s_i) = \mathit{true}$ implies $P(c) = \mathit{true}$ for all $c \in \gamma(s_i)$. □

Backward obligation slice. Given the transition witness w, the *backward obligation slice* is

$$S = \{i < w \mid \exists\, r \in \mathit{regs}(P).\ \text{instruction } i \text{ writes to } r\}$$

i.e., all instructions before w that write to registers appearing in the obligation predicate P. When `mark_precise` backtracking information is available in the trace, S is refined to include only instructions on the backtracking chain, yielding a tighter slice.

Generalization beyond eBPF. Abstract state transition analysis applies to any abstract interpreter that outputs per-step abstract states. The requirements are: (1) per-step abstract states are emitted, (2) safety properties are expressible as predicates over the abstract domain, and (3) the trace is ordered. The key insight is domain-independent: given a sequence of abstract states $[s_0, \ldots, s_n]$ and a safety predicate P, detecting the transition (s_{w-1}, s_w) where P changes from satisfied to violated locates the proof-loss point regardless of what the abstract domain represents. Rust's borrow checker (lifetime predicates over borrow state), WebAssembly validators (type predicates over operand stack state), and Java bytecode verifiers (type predicates over stack frame state) all satisfy these requirements—suggesting that abstract state transition analysis is a general technique for improving diagnostic quality in verified language runtimes.

3.6 BTF Source Correlation

OBLIGE maps each detected state transition event (proof established, proof lost, rejected) to its source location via BTF `line_info` annotations [8]. When the verifier emits a source-location comment before an instruction (e.g., `; foo.c:42`), OBLIGE records the mapping. Multiple consecutive bytecode instructions from the same source line are merged into a single span.

The result is 3–5 source-level spans, each labeled with a proof lifecycle role and annotated with the register state change at that point. When BTF annotations are absent (34.4% of our corpus, primarily Stack Overflow cases where log fragments lack BTF context), OBLIGE falls back to bytecode-level spans (instruction indices only).

BTF coverage is a data quality issue, not a parser limitation: kernel selftest cases have 98.7% BTF coverage, confirming that the correlation works correctly when the input has BTF annotations.

3.7 Diagnostic Rendering

OBLIGE produces two output formats from the same state transition analysis:

Human-readable Rust-style output. Modeled on Rust compiler diagnostics [12], the output shows multiple source spans with causal labels, register state changes at each transition, and `note`/`help` annotations. The format is designed to be readable in a terminal, an IDE, or a CI log without any tool support.

Structured JSON. A machine-readable format containing `error_id`, `taxonomy_class`, `proof_status`, an array of `spans` (each with role, source location, instruction index, and state change), the instantiated obligation predicate, and `note`/`help` text. This format is designed for consumption by LLM agents (replacing raw log text in prompts), CI systems (structured failure annotations), and IDE plugins.

3.8 Implementation Notes

OBLIGE is implemented in Python (~14,613 lines across five modules: `log_parser`, `trace_parser`, `proof_engine`, `source_correlator`, and `rust_diagnostic`), with 268 unit tests covering obligation inference, predicate evaluation, backward slicing, and transition witness detection on hand-crafted and real-corpus traces. It defines 23 error IDs (OBLIGE-E001 through E023) and nineteen obligation families.

Source-code independence. OBLIGE requires only the verifier log text as input—no object files, source files, or external debug information. When BTF `line_info` annotations are present in the log, OBLIGE maps proof events to source-level spans; when BTF is absent, it falls back to bytecode-indexed spans. For already-loaded programs, `bpftool prog dump xlated linum` can supply supplementary source context, but this is optional.

Language independence. Because OBLIGE analyzes the verifier trace at the BPF bytecode level, it is agnostic to the source language. Table 2 breaks down diagnostic performance by source language across all 302 cases (including ineligible ones). Diagnostic success is uniformly high: 85.8% for C, 95.2% for Rust (Aya), and 100% for Go (Cilium). The lower obligation inference rate for Rust (40.0% vs. 93.2% for C) and the near-zero BTF coverage for non-C cases reflect the log richness available in the corpus (Rust and Go cases are primarily from GitHub issues with truncated logs), not a language-specific limitation in the parser.

4 Evaluation

We evaluate OBLIGE against six questions:

Table 2. Per-language Oblige diagnostic performance (302 cases).

Lang.	Cases	Diag Success	Obligation	BTF
C	274	235 (85.8%)	219/235 (93.2%)	170 (62.0%)
Rust	21	20 (95.2%)	8/20 (40.0%)	1 (4.8%)
Go	7	7 (100%)	5/7 (71.4%)	1 (14.3%)

Table 3. Evaluation corpus.

Source	Cases	Eligible	BTF	Known fix
Kernel selftests	200	171	98.7%	200
Stack Overflow	76	65	1.5%	66
GitHub issues	26	26	7.7%	15
Total	**302**	**262**	**65.6%**	**281**

Q1 Does Oblige generate diagnostics reliably at scale?
Q2 What fraction of cases get full proof obligation coverage?
Q3 Do Oblige's source spans cover actual fix locations?
Q4 Do Oblige diagnostics improve LLM-assisted repair?
Q5 How does Oblige compare to Pretty Verifier?
Q6 What is the runtime overhead?

4.1 Corpus and Methodology

Table 3 describes our evaluation corpus. The primary corpus comprises 302 real verifier failure cases from three sources: 200 kernel selftests (with `__msg()` annotations specifying expected verifier errors), 76 Stack Overflow questions with attached verifier logs, and 26 GitHub issues from Cilium, Aya, and Katran. Of the 302 cases, 262 have verifier logs of sufficient length for diagnostic evaluation (≥50 characters); these form our evaluation corpus.

Thirty cases are manually labeled with ground-truth taxonomy by a domain expert (the first author), achieving Cohen's $\kappa = 0.652$ against the heuristic classifier. The heuristic classifier agrees with manual labels in 76.7% of cases overall, but only 33.3% on lowering artifacts—confirming that this is the hardest class to diagnose from error messages alone.

The taxonomy distribution across the 262-case evaluation corpus: source_bug 48.5%, env_mismatch 31.3%, lowering_artifact 12.6%, verifier_limit 7.6%.

4.2 Batch Reliability and Obligation Coverage (Q1, Q2)

We ran Oblige on all 262 eligible cases. Table 4 reports the results.

Oblige generates a structured diagnostic for every case with zero crashes. Proof obligation coverage of 94.3% (247/262) means that for nineteen in twenty cases, Oblige can state the exact predicate the verifier needed and evaluate it at every instruction. The remaining 6% involve environment errors (Permission denied, build failures) rather than true verifier rejections, and fall outside the nineteen currently supported obligation families. We explicitly characterize this scope limitation in Section 5.

Table 4. Batch diagnostic evaluation on 262 cases (Q1, Q2).

Metric	Result
Diagnostic generation success	262/262 (100%)
Proof obligation inferred	247/262 (94.3%)
BTF source correlation	172/262 (65.6%)
Proof-established span	115/262 (43.9%)
Proof-lost span	99/262 (37.8%)
Causal chain extracted	24/262 (9.2%)
Proof status: never_established	129/262 (49.2%)
Proof status: established_then_lost	99/262 (37.8%)

The proof status distribution reflects the failure mix: 49.2% are *never_established* (source bugs with no proof at any point), 37.8% are *established_then_lost* (proofs that exist in source but are destroyed during lowering or across program structure). This 37.8% is the population where Oblige provides its strongest differentiating value. Of these, 24 cases (9.2%) have structured causal chains extracted via `mark_precise` backward slicing (Section 3.5), connecting the proof-loss instruction to the final rejection through register-level dependency edges.

Span coverage (Q3). For cases with known fixes, 12/14 manually annotated cases (85.7%) have at least one Oblige span overlapping the fix location; on kernel selftests, 85/102 (83.3%) of rejected spans match the expected error instruction. The overall 38.4% (101/263) figure is deflated by Stack Overflow cases where fix locations are described in prose rather than as line numbers—a corpus limitation, not a parser one.

4.3 LLM Repair Experiment (Q4)

We evaluate whether Oblige diagnostics help an LLM generate better repairs. We use an A/B design: the same LLM (GPT-4.1-mini) receives either (A) buggy code plus raw verifier log, or (B) buggy code plus raw log plus Oblige's Rust-style diagnostic. We measure fix-type accuracy (does the LLM choose the correct type of fix?), root-cause targeting (does it target the right location?), and semantic similarity to the known correct fix.

Table 5 shows results on 54 cases (10 lowering artifacts, 28 source bugs, 8 verifier limits, 8 env mismatches).

The headline overall fix-type accuracy shows Condition A marginally ahead (85.2% vs 79.6%), but the per-class breakdown tells the story. On lowering artifacts—the class where raw error messages are most misleading—Oblige diagnostics double the fix-type accuracy: 3/10 (30%) with raw logs versus 6/10 (60%) with Oblige diagnostics, a +30 pp gain. In Condition A, the LLM sees an error like "unbounded min value" and adds another bounds check (wrong). In Condition

Table 5. A/B LLM repair experiment (54 cases). Condition A: raw verifier log only. Condition B: raw log + OBLIGE diagnostic.

Metric	A (raw log)	B (+OBLIGE)	Δ
Overall location accuracy	53/54 (98.1%)	48/54 (88.9%)	−9pp
Overall fix-type accuracy	46/54 (85.2%)	43/54 (79.6%)	−6pp
Lowering fix type	**3/10 (30%)**	**6/10 (60%)**	**+30pp**
Source bug fix type	27/28 (96.4%)	23/28 (82.1%)	−14pp
Verifier limit fix type	8/8 (100%)	7/8 (87.5%)	−12.5pp
Env mismatch fix type	8/8 (100%)	7/8 (87.5%)	−12.5pp

Table 6. OBLIGE vs. Pretty Verifier on 30 manually labeled cases (top) and the full 262-case corpus (bottom).

Metric	Pretty Verifier	OBLIGE
30-case manually labeled subset		
Overall classification accuracy	19/30 (63%)	25/30 (83%)
Lowering artifact accuracy	1/6 (17%)	4/6 (67%)
Root-cause localization	0/30 (0%)	12/30 (40%)
Cases with actionable diagnosis	5/30 (17%)	20/30 (67%)
Crash-free operation	22/30 (73%)	30/30 (100%)
Full 262-case corpus		
Crash-free operation	234/262 (89.3%)	262/262 (100%)
Produces recognized output	75/262 (28.6%)	262/262 (100%)
Root-cause localization	0/262 (0%)	53/262 (20.2%)
Multi-span diagnostic	0/262 (0%)	115/262 (43.9%)
Causal chain	0/262 (0%)	24/262 (9.2%)

B, it sees that the bounds check at line 3 *established* the proof, the `htons` call at line 7 *destroyed* it, and it correctly applies a scalar clamp.

The overall regression on non-lowering classes (Condition B slightly below A) has a known root cause: several source-bug cases received OBLIGE output pointing to a BTF metadata issue rather than the actual logic fix (e.g., BTF/func_info regeneration advice on dynptr and iterator cases), pushing the LLM away from the correct repair. The raw log for these cases already named the specific contract violation, and the generic OBLIGE output added noise. This is a known issue with BTF-absent traces and is targeted for improvement.

We present this as evidence that OBLIGE's value is concentrated on the lowering-artifact class, where proof-trace analysis provides qualitatively different information not available in raw logs.

4.4 Comparison with Pretty Verifier (Q5)

We compared OBLIGE against Pretty Verifier on the 30 manually labeled cases. Table 6 summarizes the results.

The key distinction is not classification accuracy. It is root-cause localization. Pretty Verifier finds root cause in 0/30 cases because it does not analyze state evolution. OBLIGE finds an earlier proof-loss site in 12/30 cases (40%)—and in 4/4 of these cases, the proof-loss site is a *different instruction* than the final rejection, providing qualitatively different guidance.

Expanded 262-case comparison. The 30-case results generalize to the full corpus (Table 6, bottom). Pretty Verifier crashes on 28/262 cases (10.7%) and produces recognized output on only 75/262 (28.6%); the remaining 60.3% receive "Error not managed." OBLIGE processes all 262 cases with zero crashes, produces multi-span diagnostics on 115 (43.9%), and localizes an earlier root-cause instruction on 53 (20.2%). The separation is sharpest on lowering artifacts: Pretty Verifier handles 5/33 and crashes on 8/33, while OBLIGE produces multi-span output on 24/33.

Source localization also differs fundamentally. Pretty Verifier maps source via `llvm-objdump` on the compiled object file (unavailable in most deployment contexts). OBLIGE uses BTF annotations embedded in the log itself, achieving 65.6% coverage without any external files.

4.5 Runtime Overhead (Q6)

OBLIGE processes all 262 cases with median latency 25.3 ms (P95: 41.2 ms, max: 89.3 ms, scaling linearly with log length at Pearson $r = 0.802$). All cases complete in under 100 ms—comfortably within interactive bounds and negligible compared to a single LLM API call (1–10 s).

5 Discussion and Limitations

5.1 Scope and Honest Limitations

94% obligation coverage. Nineteen obligation families cover 94.3% of cases. The remaining 6% involve environment errors (Permission denied, build failures) rather than true verifier rejections; these fall outside the scope of proof-trace analysis. The predicate framework is extensible, and adding a new family requires defining the predicate template and the parameter extraction rules. We provide an explicit enumeration of the nineteen supported families (Table 1) so readers can assess scope.

BTF coverage. 34.4% of cases lack BTF annotations (primarily Stack Overflow fragments). Without BTF, OBLIGE produces bytecode-level spans (instruction indices) rather than source-level spans. This degrades human readability but not the underlying predicate analysis. The 98.7% BTF coverage on kernel selftests confirms that the correlation is correct when inputs are well-formed.

Path-insensitive slicing. OBLIGE's backward slice from the transition witness is bounded and path-insensitive. It will not correctly handle all loop- or alias-sensitive proof loss patterns. We scope the slice as an explanatory aid after the main predicate/transition detection, and validate it empirically rather than claiming formal precision.

LLM repair experiment uses proxy metrics. The A/B experiment uses 54 cases and proxy metrics (fix-type classification) rather than verifier-pass outcomes. The regression on source-bug cases caused by BTF-absent traces pointing toward metadata issues is a known failure mode. A full re-run with updated prompts and a verifier-pass oracle is planned.

No cross-kernel stability evaluation. The verifier's log format varies across kernel versions. We have not yet evaluated Oblige across multiple kernel versions. Prior work [15] finds that 83% of eBPF extensions are affected by at least one kernel mismatch; this is an important axis for future work.

5.2 Upstream Potential

Oblige's output could become part of the kernel's own diagnostic infrastructure. The structured JSON format and 23 error IDs are designed to be stable and extensible. The proof obligation catalog could inform improvements to verifier error messages. The Rust-style rendering could be integrated into `bpftool` or the `libbpf` loader, so developers receive structured diagnostics directly from the load path.

6 Related Work

6.1 eBPF Verifier Complexity and Developer Pain

Jia et al. [7] argue that kernel-extension verification is "untenable" as the extension surface grows; Oblige addresses the near-term reality that millions of C/Rust eBPF programs run against the current verifier. Rex [6] studies 72 verifier workaround commits, confirming 44.4% are LLVM code-generation changes; Rex explains *why* such failures exist, Oblige explains *this specific failure instance* at load time. Deokar et al. [2] find 19.3% of 743 Stack Overflow questions are verifier-related, quantifying the pain point Oblige addresses.

6.2 Diagnostic Tools

Pretty Verifier [9] applies 91 regex patterns to the final verifier error line, producing a single-span explanation; it cannot detect proof-loss transitions and crashes on 10.7% of our corpus. Oblige analyzes abstract state transitions across the complete execution trace—a different level of analysis that enables root-cause localization and lowering artifact diagnosis. The `ebpf-verifier-errors` repository [11] crowdsources (log, fix) pairs as data, but provides no automated analysis.

6.3 LLM-Based eBPF Tools

Kgent [16] feeds raw verifier output verbatim to an LLM repair loop; Oblige's structured diagnostics could serve as Kgent's preprocessor, replacing unstructured register dumps with labeled causal spans. SimpleBPF [3] generates a high-level DSL and explicitly notes that raw-log retry "was not effective"; Oblige addresses the general case (arbitrary hand-written C/Rust) that DSLs cannot replace. K2 [14] optimizes eBPF programs for performance, a goal orthogonal to failure diagnosis.

6.4 Static Analysis and Diagnostics

Oblige's approach is inspired by counterexample analysis in model checking: reconstruct why verification failed from the evidence trace. Abstract interpretation [1] and its formalization for eBPF [4] ground Oblige's state transition analysis. Program slicing [13] motivates backward dependency extraction from the transition witness. Rust's compiler diagnostics [12] pioneered multi-span source-level errors with causal labels; research on compiler error messages [5] confirms that pointing at the wrong location significantly harms developer productivity.

7 Conclusion

The eBPF verifier already emits everything needed to understand why a program is rejected—but that information is buried in verbose, thousand-line traces. Oblige demonstrates that a simple yet powerful idea, analyzing abstract state transitions in the verifier's execution trace, can transform these opaque logs into precise, actionable diagnostics. The clearest payoff is for compilation-induced failures, where existing tools localize root causes in 0% of cases; Oblige pinpoints the proof-loss point in 67% and boosts automated repair accuracy by 30 percentage points. At 25.3 ms median latency, entirely in userspace, and with no kernel modifications, Oblige is practical for integration into existing eBPF development workflows. As eBPF programs grow in complexity and the verifier continues to evolve, abstract state transition analysis offers a principled, generalizable path to making verification failures comprehensible.

References

[1] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. *SIGPLAN Notices* (1977), 238–252. https://doi.org/10.1145/512950.512973

[2] Mugdha Deokar, Jingyang Men, Lucas Castanheira, Ayush Bhardwaj, and Theophilus A. Benson. 2024. An Empirical Study on the Challenges of eBPF Application Development. In *Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions*. 1–8. https://doi.org/10.1145/3672197.3673429

[3] Xiangyu Gao, Xiangfeng Zhu, Bhavana Vannarth Shobhana, Yiwei Yang, Arvind Krishnamurthy, and Ratul Mahajan. 2025. Offloading the Tedious Task of Writing eBPF Programs. In *Proceedings of the 3rd Workshop on eBPF and Kernel Extensions*. 63–69. https://doi.org/10.1145/3748355.3748369

[4] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 1–15. https://doi.org/10.1145/3314221.3314590

[5] Brett Hartmann, Mark Deline, and Peter Lee. 2014. Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research. In *Proceedings of the Workshop in Primary and Secondary Computing Education (WiPSCE)*. 75–84. https://doi.org/10.1145/2670757.2670780

[6] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, 325–342.

https://www.usenix.org/conference/atc25/presentation/jia

[7] Jinghao Jia, Raj Sahu, Adam Oswald, Dan Williams, Michael V. Le, and Tianyin Xu. 2023. Kernel Extension Verification is Untenable. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HotOS '23)*. 150–157. https://doi.org/10.1145/3593856.3595892

[8] Linux Kernel Documentation. 2024. BPF Type Format (BTF). Linux kernel documentation. https://docs.kernel.org/bpf/btf.html

[9] Sebastiano Miano, Roberto Lezzi, Gabriele Lospinoso, and Fulvio Risso. 2025. Pretty Verifier: Towards Friendly eBPF Verification Errors. GitHub repository. https://github.com/netgroup-polito/pretty-verifier Unpublished, accessed 2026-03-12.

[10] NCC Group. 2024. *eBPF Verifier Code Audit*. Technical Report. The Linux Foundation. https://www.linuxfoundation.org/press/linux-foundation-releases-open-source-technology-security-audit-of-the-ebpf-verifier

[11] parttimenerd. 2024. ebpf-verifier-errors. GitHub repository. https://github.com/parttimenerd/ebpf-verifier-errors Accessed 2026-03-12.

[12] Rust Language Team. 2016. Improved Error Messages in the Rust Compiler. The Rust Blog. https://blog.rust-lang.org/2016/08/10/Shape-of-errors-to-come.html Accessed 2026-03-12.

[13] Mark Weiser. 1981. Program Slicing. In *Proceedings of the 5th International Conference on Software Engineering (ICSE)*. 439–449.

[14] Qiongwen Xu, Michael D. Brown, Tao Li, Chengbin Pang, Bhavna Sopori, Sahil Jain, Junfeng Yang, and Arpit Gupta. 2023. K2: Automated eBPF Program Optimization. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 1–15. https://doi.org/10.1145/3603269.3604882

[15] Junxiao Zhang, Samantha Mathews, Hiep Nguyen, Ryan Stutsman, and Asaf Cidon. 2025. DepSurf: Measuring and Analyzing the Dependency Surface of Kernel Extensions. Project site and paper. https://sites.google.com/view/depsurf-bpf/home

[16] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions*. 30–36. https://doi.org/10.1145/3672197.3673434